PYTHON FOUNDATIONS: DATA, FUNCTIONS, AND FILES

Values, names, and types

Python programs manipulate objects. A variable name refers to an object; the name does not permanently acquire the type of its first value. Assigning score = 90 and then score = "ninety" is allowed, although changing the meaning of a name can make a program difficult to understand. Common built-in types include integers, floating-point numbers, strings, booleans, lists, tuples, sets, dictionaries, and None. Choose types that reflect the meaning of the information rather than storing everything as strings.

Input from input() arrives as text. Converting that text with int() or float() can fail when the user supplies an unsuitable value. Catch ValueError at the point where conversion is expected to fail and explain what input is required. Do not wrap an entire program in a broad exception handler that silently continues. A missing file, a malformed number, and a programming mistake require different responses. Precise handling makes the program easier to debug and gives users a useful next step.

Strings represent text and are immutable. Methods such as strip(), lower(), and replace() return new strings. They do not alter the original object. Use strip() when leading and trailing whitespace has no meaning, such as validating a question in a chatbot. Preserve whitespace when it carries information, as it can in code samples. A question that contains only spaces should be treated as empty after trimming. Validate before performing expensive work such as generating an embedding or calling a language model.

Collections and mutation

A list is an ordered, mutable sequence. It suits a history of chat messages because messages have an order and new items are appended. A tuple is an ordered sequence that is immutable, making it useful for fixed groups of values. A set stores distinct hashable items and supports membership checks and set operations. A dictionary maps keys to values. In a document application, a dictionary can represent metadata with keys such as source and chunk_index. Clear field names make these records easier to inspect.

Mutable objects can be shared by multiple names. If two variables refer to the same list, appending through one variable is visible through the other. This matters when storing conversation state or creating default arguments. Do not use a mutable list as a function's default when each call should receive a new list. Use None as the default and create the list inside the function. For Pydantic fields containing lists, a default factory makes the intention explicit and avoids accidental shared state.

Dictionary lookup with data["key"] raises KeyError when the key is missing. Lookup with data.get("key") can return a default instead. Use direct access when a missing key indicates an invalid contract; use get() when the field is optional. An API adapter should validate the shape of external data before passing it into the rest of the program. For example, an embedding response must contain a list of vectors with the expected number of items. Checking only that the HTTP status is successful is insufficient.

Control flow and reusable functions

An if statement selects a branch based on a condition. A for loop visits items in an iterable. A while loop continues until its condition becomes false. When processing long text in overlapping windows, ensure that the next starting position always moves forward. Otherwise a poorly chosen overlap can create an infinite loop. Validate that overlap is nonnegative and smaller than chunk size. Also stop when the final character has been included, rather than creating unnecessary repeated chunks at the end of a document.

A function should perform a clearly named task. A chunk_text function can receive text, size, and overlap and return a list of strings without reading files or making network calls. Keeping this transformation separate makes it easy to test with empty text, short text, long paragraphs, and unusual whitespace. A separate ingestion function can handle reading files and storing chunks. Another function can build a model prompt. These boundaries reduce the amount of unrelated work that must be understood to change one behavior.

Type hints document intended inputs and outputs. A declaration such as def count_words(text: str) -> int helps readers and analysis tools, but ordinary Python type hints do not automatically validate every value at runtime. Validate data at system boundaries with explicit checks or a validation library. A docstring should explain a function's purpose and any important behavior, such as whether indexing removes deleted documents. Avoid comments that simply repeat the next line of code without explaining a decision.

Paths, files, and text encodings

Use pathlib.Path to represent filesystem locations. Building paths with directory / "filename.txt" is clearer and more portable than manually joining strings with slashes. Relative paths are interpreted from a working directory, which can change depending on how the program is launched. A default document path derived from the module's own __file__ is predictable. Environment variables can override that path for a container or a developer's computer. Keep configuration choices separate from the code that reads and processes documents.

Read corpus files with an explicit UTF-8 encoding. The file extension does not guarantee its encoding, and silently replacing undecodable characters may damage important content. For an educational chatbot, a clear ingestion error is preferable to quietly indexing corrupted text. A context manager closes an opened file even when processing raises an exception. Path.read_text() is convenient for small files, while streaming line by line is more suitable when files may be too large to fit comfortably in memory.

Deterministic behavior and practical checks

Sorting discovered filenames gives ingestion a predictable order. A cryptographic hash of a source name, chunk index, and chunk text can serve as a stable record identifier. The same document produces the same identifiers when chunking settings do not change. This supports idempotent indexing: repeating an ingestion request should preserve the total number of records rather than creating duplicates. Changed content creates new identifiers, and an explicit synchronization step can remove records that are no longer present in the corpus.

A practical Python workflow separates source code, environment configuration, dependencies, and generated data. Put installed packages in a virtual environment, list application dependencies in requirements.txt, and exclude the environment from Git. Keep persistent application data out of the source repository. Use meaningful tests to verify user-visible behavior, such as rejecting blank questions or preserving indexed documents across service reconstruction. Passing those tests establishes specific evidence; it does not automatically prove that Docker networking or a real language model works on another computer.

HTTP AND FASTAPI: BUILDING A SMALL DOCUMENT API

Requests, responses, and endpoints

HTTP is a request-response protocol used by web applications and many service APIs. A client sends a request containing a method, a path, headers, and sometimes a body. A server returns a status code, headers, and usually a response body. An endpoint is a particular method and path that the server handles, such as GET /stats or POST /ask. The same path can have different behavior for different methods. Choosing clear endpoint names helps a frontend call the service without knowing its internal implementation.

GET typically retrieves information without requesting a state change. POST commonly submits data or starts an operation. A document chatbot can expose GET /health to report dependency status, GET /stats to report index size, POST /ingest to synchronize documents, and POST /ask to answer a question. A root endpoint at GET / is a convenient small check that the HTTP application is running. Document ingestion changes stored state, so it should be an explicit POST operation rather than an accidental side effect of visiting a status page.

JSON and data contracts

JSON represents objects, arrays, strings, numbers, booleans, and null. It is a common format for API requests and responses because many programming languages can read it. A question request can contain a single question field. A useful answer response contains answer, sources, confidence, and chunks_retrieved fields. Source entries should identify the original filename and chunk index so that a reader can inspect the evidence. Consistent field names let the interface render a response without guessing which keys are available.

FastAPI uses Python type annotations and Pydantic models to define and validate data contracts. A Pydantic question field can strip surrounding whitespace and require a minimum length of one character. Both an empty string and a string containing only spaces should be rejected before reaching the RAG pipeline. A maximum length prevents unexpectedly large questions from consuming excessive time and context. Validation failures normally produce an HTTP 422 response with structured details explaining which input failed the contract.

Response models also matter. They document the fields returned by an endpoint and help catch mismatches between implementation and public behavior. Distinguish document_count from chunk_count. If six files each produce ten chunks, document_count is six and chunk_count is sixty. Calling both values documents would confuse users and make persistence checks unreliable. A confidence field should use a small documented vocabulary such as high, medium, and low. Explain whether that vocabulary reflects retrieval distance, answer correctness, or some other measurement.

Status codes and failure handling

A 200 response means that the HTTP request succeeded according to the endpoint's contract. A 400 response can indicate that an ingestion request cannot proceed because no supported text files were found. A 422 response indicates invalid request data. A 503 response is appropriate when a required service such as Ollama is unavailable. A 500 response indicates an unexpected server failure. Error messages should give useful next steps while avoiding enormous internal stack traces in the public JSON response. Detailed exceptions belong in application logs.

Liveness and readiness answer different questions. Liveness asks whether the application process can respond. Readiness asks whether the dependencies required for useful work are available. A /health endpoint can return 200 while reporting status = degraded and ready = false when Ollama is disconnected. This lets an operator inspect the problem through the API. A Docker healthcheck must inspect the ready field rather than assuming that any 200 response means the whole application is ready. Documenting this distinction prevents misleading green status indicators.

Dependency boundaries and lifecycle

Dependency injection supplies a route with the service it needs. Instead of constructing a new database client inside every endpoint, create a long-lived RAG service during application lifespan and resolve it from application state. Tests can supply a separate service configured with a temporary database path and a controlled HTTP adapter. This arrangement exercises the actual routes while preventing tests from touching a user's real index or downloading models. Keep dependency creation separate from the endpoint's request validation and response construction.

An application lifespan context can initialize resources when the server starts and close them when it stops. Startup ingestion should not happen merely because a test imports a Python module. Use lifespan startup for that work. Synchronous file reading, vector operations, and HTTP calls can block; run them in synchronous route functions or an appropriate worker thread instead of directly blocking an asynchronous event loop. A minimal embedded Chroma application should use a single server process unless its storage design explicitly supports a different deployment pattern.

CORS and service addresses

Cross-Origin Resource Sharing is a browser mechanism that controls whether frontend JavaScript may access responses from another origin. An origin includes scheme, host, and port. A FastAPI CORS middleware configuration can allow the specific origins used during development. CORS is not authentication and should not be presented as protection for a publicly exposed API. Streamlit typically makes this project's backend requests from its Python server, so those calls are server-to-server and are not governed by the browser's CORS checks in the same way.

An address meaningful inside a container network may not resolve in a Windows browser. The frontend container can call http://backend:8000 because backend is a Compose service name. A person usually opens the published host address http://localhost:8501 to reach the interface. The backend container calls Ollama through the configured Ollama service address. Keep these addresses in environment configuration so the same Python modules can be used locally and in containers. A host port change does not automatically change the service's internal listening port.

Interactive documentation and verification

FastAPI generates interactive API documentation, commonly available at /docs. Swagger UI allows a developer to expand an endpoint, enter a request, execute it, and inspect the response. A useful manual sequence is to check health, inspect stats, ingest documents, ask an answerable question, and ask a question outside the corpus. Also submit a blank question to confirm validation. Automated endpoint tests should verify the same public contracts, but a passing mocked HTTP test does not establish that a real Ollama model is downloaded or that containers can reach one another.

STREAMLIT CHAT INTERFACES AND SESSION STATE

The rerun execution model

Streamlit builds an interface by executing a Python script. When a user interacts with many widgets, Streamlit reruns the script from top to bottom. A local variable created during one run is not a reliable place to preserve conversation history for the next run. Session state supplies a per-session mapping that survives these reruns. Initialize a messages list only if it is absent from session state. Replacing it unconditionally on every run would erase the conversation each time the user submits a question or clicks a sidebar button.

Session state is associated with a user's application session. It is not a permanent database and should not be described as durable chat storage across all browser restarts or server restarts. For this minimal chatbot, preserving history across widget reruns meets the interface requirement. A separate persistence design would be needed to save conversations permanently or share them between devices. Keep this distinction visible in documentation so users understand which information remains after the application is closed and which information is preserved by a Docker volume.

Chat elements and message records

st.chat_message() creates a chat message container for a user or assistant. st.chat_input() displays a place for the user to submit a question. A simple rendering loop reads saved message records and displays each inside a container for its role. A message record can contain role, content, confidence, and sources. User messages generally need only role and content. Assistant messages can carry the retrieved source excerpts so the same evidence remains visible when Streamlit reruns the script after a later interaction.

When the user submits a question, first append it to session state and display it. Then call the backend, construct an assistant message from the response, append that message, and render it. The next rerun will reconstruct the entire conversation from the saved records. Do not append the same message once during processing and again during the history-rendering loop. The rendering loop should only display records. Separating state updates from rendering makes duplicate messages easier to prevent and makes interface tests more predictable.

Showing evidence rather than only an answer

A source citation is useful only if the reader can connect it to actual evidence. An answer containing [S1] should have a source entry labeled S1. Use an expandable section to keep the main conversation readable while allowing inspection of the filename, chunk index, retrieval distance, and full excerpt. The index refers to a chunk within the source file, not necessarily a printed page number. Display the source text as plain text when possible so unusual formatting in a document is not confused with interface controls or instructions.

Confidence labels need a clear explanation. In this project, confidence comes from the best retained cosine distance, which measures how close the question embedding is to a retrieved chunk embedding. High retrieval confidence does not prove that every generated statement is correct. A low-confidence fallback should explain that the corpus does not supply enough evidence. Never invent a percentage of correctness just to make the interface look precise. A short caption and an inspectable source panel can communicate more honestly than an unsupported numerical confidence score.

Sidebar controls and feedback

The sidebar is a convenient place for status information that should remain separate from the conversation. Show whether the document store and Ollama are connected, whether configured models are available, and how many source documents and chunks are indexed. A Re-index Documents button calls POST /ingest. Re-indexing should reflect changed or removed files without accumulating duplicate chunks. A Refresh status button lets a user retry after a service finishes starting. A Clear conversation button should remove session messages without deleting the document database or downloaded models.

Long operations need visible feedback. A spinner can explain that the application is updating the index or generating an answer. Show a success message after a completed re-index, then refresh the counts. Preserve that success message across an intentional rerun by placing it temporarily in session state. Handle backend failures with concise, actionable text. For example, explain that a model may still be warming up after a timeout. Do not replace every failure with an empty answer, because an empty message hides the difference between a model refusal, a network error, and a missing index.

Calling a backend service

Keep the backend's base URL in an environment variable such as BACKEND_URL. The interface should append endpoint paths to that configured base rather than hardcoding container service addresses throughout the script. A small request helper can apply timeouts, check HTTP status codes, decode JSON, and convert expected failures into user-facing messages. Health requests should have shorter timeouts than ingestion and generation requests. A status panel should not appear frozen for several minutes just because the language model service is unavailable.

An HTTP timeout limits how long the client waits, but it does not guarantee that the server canceled the computation. If a generation request times out, the backend may still be working. Avoid automatically retrying state-changing or expensive requests indefinitely. Let the user choose when to ask again. Ingestion may be safe to repeat when it is idempotent, but repeated work can still waste time. A clear architecture separates interface state, API transport, and the backend's document logic so each part can be verified independently.

Caching and application testing

Caching can reduce repeated expensive work, but choose cached data carefully. A cached health response can briefly show a service as available after it stops. Cached index counts can remain stale after re-indexing unless the cache is cleared or its lifetime expires. For a small local application, fresh lightweight status requests on reruns are a reasonable choice. Do not cache all users' chat histories in a global resource. The history belongs in per-session state, while shared resources such as a reusable connection pool require different lifecycle considerations.

Streamlit's AppTest can execute an interface script and inspect its elements without a full browser. A meaningful test can mock HTTP responses, submit a chat question, and verify that an assistant answer, confidence caption, and source expander appear. Another test can trigger a rerun and confirm that conversation history remains. A failure-state test can make the backend unavailable and confirm that the application displays a useful message instead of crashing. These checks supplement a real browser walkthrough, especially for accessibility, layout, and the complete container networking path.

RETRIEVAL AUGMENTED GENERATION WITH CHROMADB

What the pipeline does

Retrieval augmented generation, usually abbreviated RAG, combines document search with a language model. The application first finds document passages relevant to a question. It then supplies those passages as context while asking the model to write an answer. The model is not retrained on the corpus during this process. Adding a document changes the searchable index, while changing a prompt changes the instructions used for a generation request. Keeping these operations separate makes it easier to update knowledge and diagnose why an answer was unsupported.

A minimal pipeline has three stages: ingest, retrieve, and generate. Ingestion loads files, splits them into chunks, computes embeddings, and stores records. Retrieval embeds the user's question and searches for nearby chunk vectors. Generation builds a prompt containing system instructions, labeled evidence, and the question, then calls the local model. A useful response includes answer text, source details, and a confidence label. The application should explicitly abstain when relevant evidence is missing rather than treating any model-generated text as a successful grounded answer.

Chunking documents

A chunk is a bounded excerpt of a document. Whole documents may be too long to fit into the model's prompt or may combine several unrelated topics. Very small chunks can lose the context needed to explain an idea. A practical starting strategy is to prefer paragraph boundaries while limiting chunks to about one thousand characters. If a paragraph is too long, split near a sentence or word boundary. A small character overlap, such as one hundred fifty characters, carries some context from one chunk into the next.

Character counts are not token counts. A thousand characters of prose and a thousand characters of source code can tokenize differently. The embedding model's input limit and the chat model's context limit are separate constraints. Limit chunk size, the number of retrieved chunks, question length, and answer length together. Do not silently truncate embedding inputs if doing so would change what part of a chunk is searchable. Overlap also consumes storage and prompt space, so increasing it indefinitely is not a reliable way to improve retrieval quality.

Embeddings and compatible spaces

An embedding is a numerical vector representing aspects of an input's meaning. Documents and queries must be embedded in the same compatible space. Mixing vectors from unrelated embedding models can produce meaningless similarity comparisons or dimension errors. Use the configured embedding model for both ingestion and question retrieval. Some embedding models distinguish document and query tasks through input prefixes. A configuration change to the model or its preprocessing should create a separate collection or trigger a deliberate re-index rather than reusing incompatible stored vectors.

Ollama can serve both a chat model and an embedding model, but those are different jobs. In this project, llama3.2:1b generates answer text and nomic-embed-text:v1.5 generates embeddings. An embedding model returns vectors rather than conversational answers. The backend sends bounded batches of document text to the embedding endpoint and sends one question embedding during retrieval. Keeping both operations local avoids relying on a paid hosted inference API. Initial image and model downloads still require network access before an offline workflow is possible.

Storing records in ChromaDB

ChromaDB stores vectors together with identifiers, document text, and metadata. Each chunk should include its source filename and its chunk index. A stable identifier can be derived from the filename, index, and content hash. Repeating ingestion then reuses unchanged records instead of adding duplicates. Upsert inserts a new identifier or updates an existing one. Synchronization should also remove identifiers for changed or deleted chunks after replacement embeddings are available. If reading the corpus or creating embeddings fails, preserve the previous working index when possible.

The persistent Chroma client writes index data to a configured filesystem path. In a container, mount a named volume at that path so removing a container does not remove its stored records. Persisting the Chroma directory does not preserve a model downloaded somewhere else; Ollama needs its own model volume. The number of stored chunks is not the number of source files. To report a true document count, count distinct source filenames in metadata. This distinction also helps identify accidental duplicate ingestion during a persistence test.

Similarity, thresholds, and confidence

Cosine similarity measures the alignment of two vectors. Cosine distance is one minus cosine similarity, so smaller distance means closer vectors. For this project's cosine collection, a distance near zero indicates a close match, while larger values indicate weaker matches. Query for a small top-k set, then retain only results below a configured distance threshold. If the collection contains fewer than k chunks, request the available count. Avoid requesting an empty search or assuming there will always be three valid results.

A threshold is a practical retrieval guardrail, not a universal constant. The best cutoff depends on the embedding model, chunking strategy, document domain, and evaluation questions. This project starts with a maximum retained distance of 0.55 and a high-confidence cutoff of 0.30. A retained best match below the high cutoff receives high retrieval confidence; other retained matches receive medium confidence. No retained context produces a low-confidence fallback. These labels summarize retrieval strength. They are not calibrated probabilities that the generated answer is factually correct.

Grounded prompts and citations

The system prompt should require answers based only on provided excerpts, prohibit fabricated facts and citations, and ask the model to acknowledge missing information. Give each retained excerpt a short label such as [S1]. Include the original filename and text in an explicitly delimited context block. Place the question separately so the model can distinguish the user's task from reference material. Documents are untrusted data: an excerpt that says to ignore prior instructions must not be promoted to a higher-priority system instruction.

After generation, basic output validation can reject answers with missing citation labels or labels that were never supplied. A response citing [S9] is invalid if the prompt contained only [S1] and [S2]. This structural check improves traceability but does not prove that a valid citation supports every sentence. A small model may still misunderstand an excerpt. Let users inspect the actual source text, and evaluate representative questions manually. Stronger deployments can add entailment checks, sentence-level evidence review, or curated expected answers when their cost is justified.

Evaluating the full path

Use several categories of questions. An answerable question should retrieve the expected document and produce a supported answer. A related but unanswered question should acknowledge that the available notes do not contain enough detail. A clearly unrelated question should trigger the no-context guardrail or an explicit abstention. Inspect retrieved chunks separately from generated text so a search failure is not confused with a generation failure. Keep these evaluation questions stable while tuning thresholds. A passing API test with controlled embeddings verifies mechanics, while real-model evaluation is still needed to assess semantic quality.

DOCKER COMPOSE, NETWORKING, VOLUMES, AND CONFIGURATION

Images and containers

A Docker image packages a filesystem and instructions for starting an application. A container is a running instance of an image with its own writable layer. Editing files on a developer's computer does not automatically change files copied into an existing image. Rebuild when application source or dependencies change, unless a deliberate development bind mount supplies the source. A Dockerfile describes how to build an image. Compose describes how several services are configured and connected. Understanding this separation prevents confusion between rebuilding an image and restarting a container.

A useful Python Dockerfile starts from a suitable Python base image, sets a working directory, copies requirements.txt, installs dependencies, and then copies application code. Copying the requirements file before other source files lets Docker reuse the dependency layer when only application code changes. Installing with --no-cache-dir avoids keeping pip's download cache inside the final image. Expose documents the intended listening port but does not publish it to the host. The application process must actually bind to the correct address and port when the container starts.

Three services in the chatbot

The chatbot uses three Compose services. The backend runs FastAPI and owns the embedded persistent Chroma client. The frontend runs Streamlit and calls the backend over HTTP. The ollama service serves the embedding and chat models. Chroma is a library used inside the backend in this design, so it does not require a fourth service. Each service has one main responsibility, and the Compose file connects their environment settings, ports, volumes, healthchecks, and startup dependencies. Keeping the service count small makes the educational stack easier to operate.

The backend must bind to 0.0.0.0 inside its container so other containers can reach it. Binding only to 127.0.0.1 inside the backend container would limit access to that same container's loopback interface. Compose service names provide network addresses on the default project network. The frontend uses the backend service name, and the backend uses the ollama service name. A container's localhost points to itself. It does not refer to the Windows host or to a different service in the Compose project.

Published ports and browser addresses

A port mapping connects a host port to a container port. Mapping host port 8501 to container port 8501 allows a browser on the computer to reach Streamlit. If the host port is already occupied, change the configured host port and recreate the affected service. The container can continue listening on its existing internal port. This project's mappings bind to the host loopback address because it is a local demonstration. Publishing services publicly would require additional access controls and operational decisions that are outside the minimal local setup.

The addresses used by people and by containers serve different purposes. A person opens the frontend's published localhost address. Streamlit's Python code calls the backend service address configured by BACKEND_URL. FastAPI calls the Ollama address configured by OLLAMA_URL. Do not replace every service address with localhost when troubleshooting; that often breaks container-to-container communication. First identify which process is making the request and which network namespace it is running in. Then verify the configured hostname and internal port for that specific connection.

Named volumes and persistence

A named volume stores data outside a container's disposable writable layer. Mount a Chroma volume at the exact directory configured by CHROMA_PATH. Mount a separate Ollama model volume at the model storage directory used by the Ollama image. Running docker compose down removes the project's containers and network but normally preserves its named volumes. Starting the project again can reuse both indexed chunks and downloaded models. The persistence test should compare document and chunk counts before and after container recreation while temporarily disabling automatic ingestion so it cannot hide data loss.

The command docker compose down -v additionally removes the project's named volumes. In this application that means losing the saved vector index and downloaded model files. It is a reset operation, not the normal way to stop the stack. A bind mount differs from a named volume: it exposes a specific host path inside a container. The corpus directory can be bind-mounted read-only so changing text files on the host makes them available to a later re-index request. The application should not need permission to edit its reference documents.

Environment configuration

Configuration belongs in environment variables rather than scattered literals. A centralized Settings class can read the Ollama URL, model names, database path, retrieval limits, distance thresholds, and debug option. Validate combinations, such as requiring overlap to be smaller than chunk size. The .env file supplies local values and Compose interpolation. An env_file entry passes values into a service's container environment. These are related but distinct mechanisms. A value written in .env is not automatically available inside every container unless the Compose configuration passes it through.

Commit .env.example with safe, documented defaults, and ignore the real .env file in Git and Docker build contexts. A downloaded starter project can include a non-secret .env for convenience, but a fresh Git clone should create it from the example. After changing container environment values, use docker compose up -d to apply the new configuration and recreate services when necessary. A plain docker compose restart reuses the existing container configuration and may not apply changed environment values. Rebuild as well if the image's source or installed dependencies changed.

Readiness and first-run setup

Starting a process does not mean it is ready to serve useful requests. Ollama can start listening before the configured model files are downloaded. A healthcheck should verify both connectivity and model availability. This project starts Ollama, downloads missing models inside that service, and marks it ready afterward. The backend depends on a healthy Ollama service and automatically indexes the corpus. The frontend waits for backend readiness. Long first-run grace periods allow model downloads and CPU embedding work to finish without prematurely reporting an ordinary slow startup as failure.

Troubleshooting begins with docker compose ps and service logs. If Docker reports a missing Docker Desktop Linux engine pipe on Windows, start Docker Desktop and wait for the engine to become available before rerunning Compose. If a model is missing, inspect Ollama logs and confirm internet access for the initial download. If a container is healthy but a browser cannot connect, check the published host port. If the backend cannot reach Ollama, inspect its configured service address. Diagnose the earliest failing dependency rather than repeatedly rebuilding every service without evidence.

TESTING, CONTINUOUS INTEGRATION, AND PRACTICAL QUALITY

What a test establishes

A test checks a specific behavior under stated conditions. A passing test is useful evidence for that behavior, but its scope matters. Testing a root endpoint through an in-process client establishes that the route returns the expected response in that test environment. It does not prove that a browser can reach the service through Docker networking. Testing a controlled language-model response establishes prompt and response plumbing. It does not prove that a downloaded model will answer every question accurately. Describe verification precisely rather than labeling the entire system production ready because a few tests passed.

Start with public behavior that matters to the user. For this API, basic tests should verify that GET / returns 200, GET /health returns the expected fields, and GET /stats includes document_count. Additional valuable tests cover blank questions, empty indexes, unavailable dependencies, idempotent ingestion, persistence, and source citations. Avoid writing tests that merely repeat implementation details line for line. A test is strongest when it would fail for a plausible regression that would otherwise affect someone using or maintaining the project.

Isolating data and external services

Tests should not modify a developer's real Chroma collection. Use a temporary directory for persistent storage and a temporary corpus containing a few clear examples. Construct application settings explicitly so a user's .env file cannot redirect a test to production data or a different service. Each test can have an isolated service instance while still exercising the real storage library. This arrangement provides stronger evidence than replacing the entire RAG pipeline with an object that always returns a fixed count or answer.

External inference calls are expensive and nondeterministic compared with ordinary unit operations. A controlled HTTP transport can return small embedding vectors and predictable chat responses without running Ollama. Choose vectors that create known cosine distances so threshold behavior is testable. The fake service should also simulate connection failures, missing models, malformed embedding responses, and invalid citations. Name these tests clearly so readers understand which parts are real and which are simulated. Keep at least one separate manual or optional integration procedure for the actual local model.

Ingestion and persistence checks

An idempotence test indexes the same corpus twice and checks that document and chunk counts do not increase. It can also verify that unchanged chunks do not trigger new embedding requests. A synchronization test changes one file or removes it while leaving other files present, re-indexes, and verifies that obsolete text is no longer returned. A failure test makes embedding generation fail during an update and checks that the previous records remain available. These tests target data integrity risks that a simple 200-status assertion would miss.

A persistence test reconstructs a service using the same database directory and confirms that records are still present without ingesting them again. The Docker version of this test should stop and recreate containers while preserving named volumes. Temporarily set automatic ingestion to false during the restart. Otherwise startup might rebuild an empty database, making lost persistence look like successful recovery. Compare both source-file and chunk counts before and after restart. Restore automatic ingestion afterward if that is the intended normal startup behavior.

Guardrails and validation tests

Input validation should reject empty and whitespace-only questions before making inference calls. Test overly long questions as well as missing fields. When no documents are indexed, /ask should return a clear low-confidence message without calling the embedding or chat model. When all retrieved distances exceed the threshold, the pipeline should abstain and skip generation. These checks make a guardrail observable. A system prompt that asks a model to behave responsibly is helpful, but a deterministic no-context branch provides a more direct guarantee about that specific failure mode.

Citation checks should reject a generated answer that cites labels absent from the supplied context. Test an answer with no citations, one with an invalid label, and one with a valid label. Also test an explicit unknown response so the application does not present it as a confident grounded answer merely because relevant chunks were retrieved. Structural citation validation has a limited purpose. It checks reference integrity, not whether every factual claim follows from the source. Real-model evaluation should still inspect the content of the answer and the cited evidence.

Frontend behavior and recoverable errors

The interface needs tests beyond the backend's API contracts. Verify that submitting a question produces a user message and an assistant message, that source excerpts appear in an expandable section, and that session history survives a rerun. A sidebar re-index action should update the displayed counts. A clear-history action should remove conversation messages without deleting the index. Simulating a backend outage should produce an understandable interface message instead of an uncaught exception. These behaviors can be checked using Streamlit's application testing tools and controlled HTTP responses.

Use reasonable timeouts on all outbound HTTP calls. Health checks should fail quickly, while initial embedding and generation calls may need more time on a CPU. Translate expected failures into consistent API status codes and interface messages. Log unexpected failures where developers can inspect stack traces. Avoid logging whole document contents, conversation histories, or environment files merely to troubleshoot a connection. A useful error message says which dependency is unavailable and what command or setting to inspect, without exposing unnecessary internal information to the user.

Continuous integration

A GitHub Actions workflow can run on pushes to main and on pull requests. A test job checks out the repository, selects a supported Python version, installs application and test dependencies, and runs pytest. A separate Docker job validates the Compose file, builds the backend and frontend images, and inspects both image names to confirm they exist. These jobs check different concerns. Test isolation avoids downloading large models in every CI run. Building images catches missing files and packaging problems that in-process tests cannot detect.

Repository configuration must match the workflow's assumptions. If .env is ignored, CI should create a safe .env from .env.example before validating Compose or importing settings that require a URL. Run tests from the documented project directory so package imports resolve predictably. Keep dependency versions consistent between local verification and image builds. The workflow file alone is not evidence of a successful hosted run. After pushing the project to GitHub, inspect the Actions tab and confirm that the test and Docker jobs complete successfully for the submitted commit.

Documentation and a reviewable handoff

A useful README explains prerequisites, first-run downloads, startup commands, browser addresses, model choices, and how to add documents. Include an architecture diagram that shows the three services and their persistent volumes. Document the chunk strategy, confidence thresholds, and limitations of small-model answers. Provide a short acceptance checklist covering citations, unknown questions, dependency errors, persistence, and CI. Record which checks were actually run. Clear instructions let another person reproduce the project and distinguish completed verification from the remaining steps that depend on their computer.